DOCUMENTACIÓN DEL CÓDIGO FUENTE Y ARQUITECTURA

System Bulls · Análisis del Firmware C++ e Interfaces Web PROGMEM

1. ESTRUCTURA DE ARCHIVOS DEL CÓDIGO FUENTE

El código fuente del sistema se distribuye en tres archivos clave dentro de la carpeta principal del firmware:

- `parqueadero_inteligente.ino`: Firmware principal C/C++. Maneja el arranque de periféricos, el bucle de eventos asíncronos, la lógica de negocio de estacionamiento y las rutas API HTTP.
- `web_admin.h`: Encabezado C++ que contiene el código completo de la interfaz web de administración (HTML5, CSS responsivo, JavaScript ES6), encapsulado como cadena literal en la memoria de programa no volátil (`PROGMEM`).
- `web_app.h`: Encabezado C++ que contiene la Aplicación Web Progresiva para clientes (PWA) con el Manifest JSON y el Service Worker para almacenamiento en caché local, también optimizado en `PROGMEM`.

2. ESTRUCTURAS DE DATOS PRINCIPALES

El sistema utiliza las siguientes estructuras C++ y contenedores de la biblioteca estándar (`std::vector`) para mantener el estado en memoria RAM con persistencia en `LittleFS`:

```
struct Cliente { String uid; // Identificador único de la tarjeta RFID (ej. "A4F12C89") String nombre; // Nombre del propietario registrado String placa; // Placa del vehículo (normalizada en mayúsculas sin guiones) String pin; // Clave de acceso a la App Cliente (4 a 8 dígitos) bool dentro; // Estado de ocupación actual (true = dentro, false = fuera) uint32_t entrada;// Timestamp UNIX de ingreso al parqueadero }; struct Registro { String nombre; // Nombre del cliente String placa; // Placa del vehículo uint32_t entrada;// Fecha/hora UNIX de ingreso uint32_t salida; // Fecha/hora UNIX de salida uint32_t costo; // Total cobrado en pesos ($ COP) }; std::vector<Cliente> clientes; // Vector dinámico de clientes registrados (máx. 50) std::vector<Registro> historial; // Vector dinámico de salidas completadas (máx. 40)
```

3. DESGLOSE DE FUNCIONES Y MÓDULOS LÓGICOS

MÓDULO LÓGICO	FUNCIONES PRINCIPALES	DESCRIPCIÓN Y ALGORITMO INTERNO
Reloj y Tiempos	`ahora()` `fijarHora()`	Calcula el timestamp real usando `esp_timer_get_time()` y un desfase offset. Sincroniza automáticamente por NTP si hay WiFi o toma la hora del navegador del cliente/admin en la primera petición HTTP.
Magnetómetro GY-271	`magIniciar()` `magLeer()` `atenderSensor()`	Escanea el bus I2C en búsqueda de las direcciones 0x0D (QMC5883L), 0x2C (QMC5883P) y 0x1E (HMC5883L). Lee vectores X, Y, Z, calcula el módulo de desviación tridimensional y aplica filtro de media móvil.
Lector RFID RC522	`atenderRFID()` `procesarTarjeta()`	Muestrea el sensor mediante la librería `MFRC522`. Si el UID es nuevo, evalúa si el cliente está registrado. Si está fuera, valida aforo e ingresa; si está dentro, calcula la estancia y procesa la salida.
Control Electromecánico	`abrirBarrera()` `atenderBarrera()`	Mueve el servo a 90° (posición abierta), inicia un temporizador de 4000 ms y retorna automáticamente la barrera a 0° sin bloquear la ejecución del programa (sin usar `delay()`).
Interfaces API REST	`apiAdminEstado()` `apiCliente()`	Endpoints HTTP que reciben peticiones AJAX/Fetch y devuelven respuestas en formato JSON serializadas mediante la librería `ArduinoJson` v7.

4. DIAGRAMA DE TRANSACCIONES Y FLUJO OPERATIVO

```
[ Lectura RFID en RC522 ] | ▼ ¿Existe UID en clientes? ──────── NO ──────► [ Guardar en tarjetaDesconocida ] | [ Enviar notificación a /admin ] SÍ | ¿Cliente está DENTRO? | | NO SÍ | | ▼ ▼ [ ¿Cupos > 0? ] [ Calcular Δt = t_salida - t_entrada ] | | [ Costo = ceil(Δt / 60) * tarifa ] SÍ NO [ Generar objeto Registro en Historial ] | | [ Guardar en LittleFS ] ▼ ▼ [ Mostrar costo en LCD y Abrir Barrera ] [Registrar] [LCD: LLENO] [Entrada ] [Abrir Servo]
```